Unidade Curricular: Fundamentos da Segurança Informática

Relatório do Projeto

Trabalho Prático #3

DATA:

MAIO 2026

ALUNO:

JOÃO NATÁLIO Nº2023205576

01. Introdução

02. Arquitectura do Ambiente de Testes

03. Fase 1 — Testes de Segurança de Aplicações Web

04. Fase 2 — Web Application Firewall (WAF)

05. Conclusões

O presente relatório documenta os testes de segurança realizados sobre a aplicação **OWASP Juice Shop v17.2.0**, no âmbito do Trabalho Prático #3 de Fundamentos da Segurança Informática. O objetivo central consiste em explorar a segurança de aplicações web seguindo as diretrizes do **OWASP WSTG v4.2** e, de seguida, implementar e validar uma solução de **Web Application Firewall** capaz de monitorizar, filtrar e bloquear tráfego HTTP malicioso.

A metodologia é de **caixa negra (black-box)**: avaliação remota sem acesso ao código-fonte, usando o **OWASP ZAP 2.17.0** como ferramenta principal. O trabalho está estruturado em duas fases — testes diretos (Fase 1) e implementação/validação do WAF (Fase 2) — cobrindo as 11 secções WSTG aplicáveis.

Introdução

Este relatório documenta, de forma sistemática, os testes de segurança realizados sobre o **OWASP Juice Shop v17.2.0** — plataforma intencionalmente vulnerável desenvolvida pela OWASP para fins de formação em segurança aplicacional.

A metodologia de **black-box testing**, definida pelo WSTG como "the art of testing a system or application remotely to find security vulnerabilities, without knowing the inner workings of the target itself", recorre a ferramentas de scanning automatizado, fuzzing e penetração manual, todas disponíveis no ambiente Kali Linux.

Após a descrição da arquitectura, apresentam-se as 11 secções WSTG para a Fase 1, seguidas das mesmas 11 secções para a Fase 2 (WAF), terminando com as conclusões.

Arquitectura

2.1 Cenário 1 — Fase 1 (Acesso Direto)

O cliente (Kali Linux com OWASP ZAP 2.17.0) acede diretamente ao servidor Juice Shop. O Firefox usa o ZAP como proxy (`localhost:8080`), intercetando e modificando todo o tráfego HTTP.

2.2 Cenário 2 — Fase 2 (Via WAF)

Todo o tráfego HTTP passa obrigatoriamente pelo WAF (porta 80) antes de chegar ao backend. O Juice Shop na porta 3000 deixa de ser acessível diretamente.

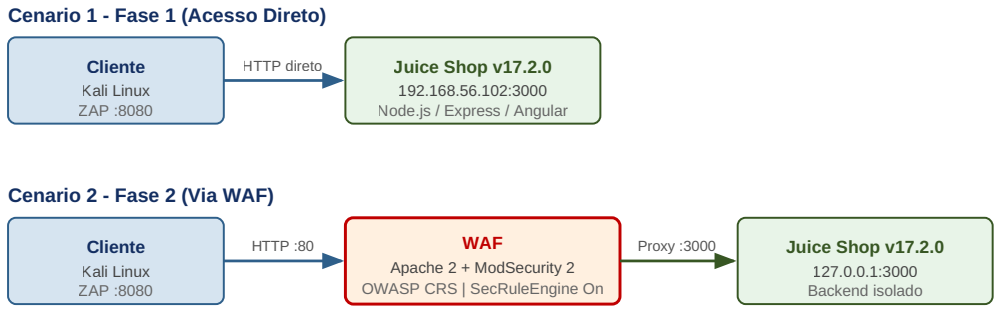


Figura 1 — Arquitectura: Cenário 1 (acesso direto) e Cenário 2 (tráfego via WAF + OWASP CRS)

Componente	Endereço / Serviço	Detalhe
Cliente — Kali Linux	OWASP ZAP 2.17.0 — proxy :8080	Firefox → ZAP → Juice Shop (F1) / WAF (F2)
Servidor — Juice Shop	192.168.56.102:3000	Node.js / Express / Angular / SQLite
WAF (Fase 2)	192.168.56.102:80	Apache 2 + ModSecurity 2 + OWASP CRS
Rede	Host-only 192.168.56.0/24	HTTP sem TLS

Departamento de Engenharia Informática | Faculdade de Ciências e Tecnologia
Universidade de Coimbra

Fase 1 — Testes de Segurança

3.1 Information Gathering (WSTG-INFO)

(a) Scan Automático — Spider

O ZAP lançou o Spider e o AJAX Spider (*If Modern* com Firefox) contra `http://192.168.56.102:3000`, descobrindo **157 URLs e 129 nós**.

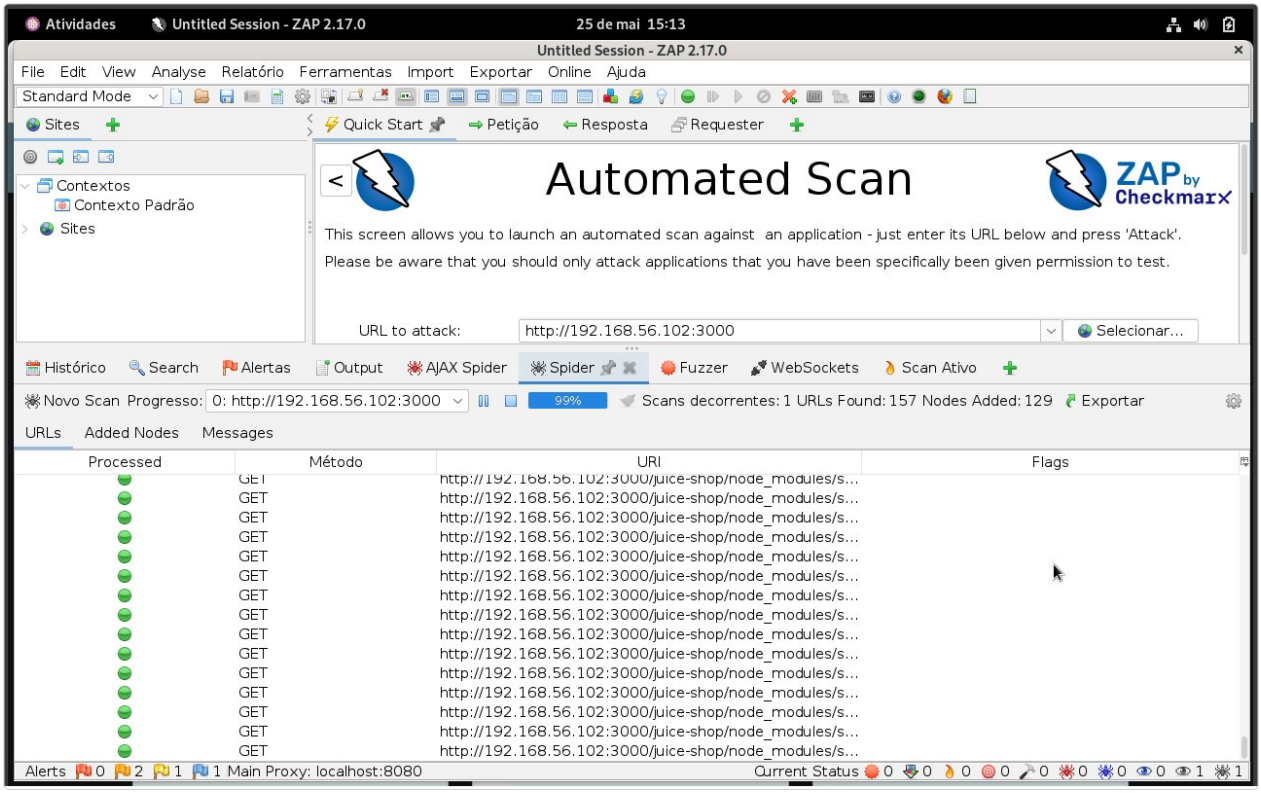


Figura 2 — Spider: 99% concluído, 157 URLs Found, 129 Nodes Added

(c) Gestão de Add-ons

Instalados no ZAP Marketplace: **FuzzDB Offensive** (v5.0 — payloads de ataque), **Technology Detection** (v21.55 — fingerprinting) e **Active Scanner Rules Beta** (v66.0 — regras adicionais).

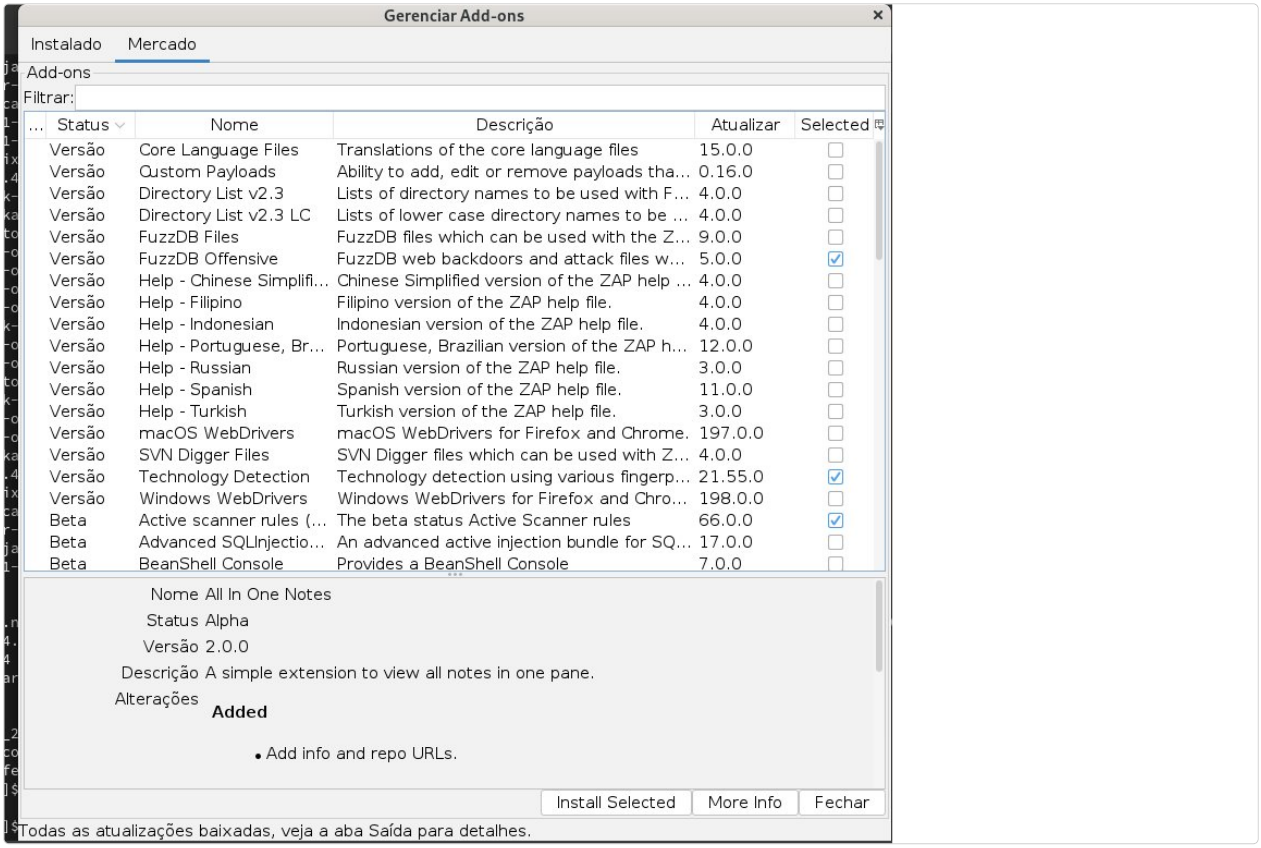


Figura 3 — Add-ons: FuzzDB Offensive, Technology Detection e Active Scanner Rules Beta selecionados

O Technology Detection identificou: Node.js/Express, Angular SPA, SQLite, WebSocket, Google Font API, SoundCloud e Gravatar. Alertas: **Private IP Disclosure** (192.168.56.102 exposto) e **Timestamp Disclosure — Unix (Systemic)**.

3.2 Configuration and Deployment Management (WSTG-CONF)

(b) Scan Ativo com Política Otimizada

Política **Default Policy**, Threshold **Baixo** e Força **Alto** — maximiza cobertura: mais ataques gerados, menor limiar de reporte.

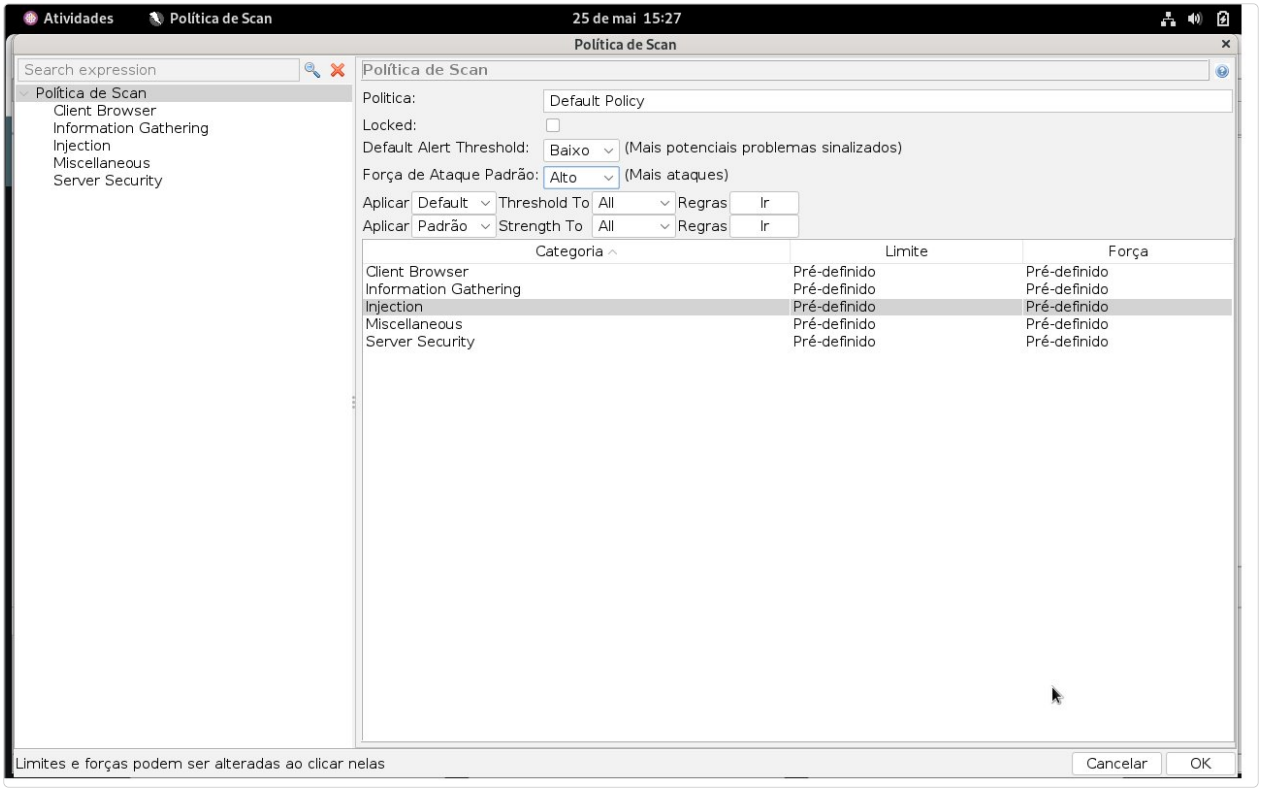


Figura 4 — Política de Scan: Threshold Baixo, Força Alto — 5 categorias ativas

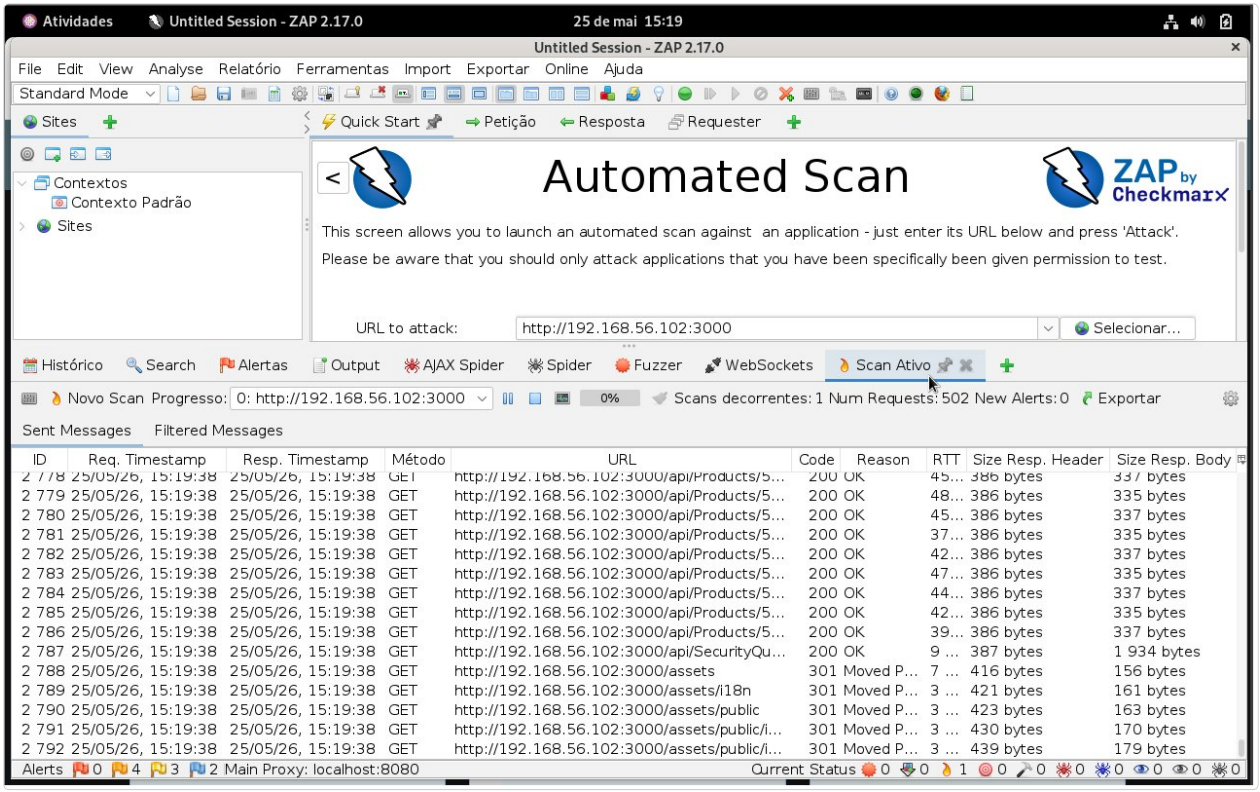


Figura 5 — Scan Ativo: 502 pedidos enviados, endpoints /api/Products e /assets testados

- **Session ID in URL Rewrite (Systemic)** — tokens em URL, visíveis em logs e cabeçalhos Referer.
- **Cross-Domain Misconfiguration (Systemic)** — CORS permissivo sem restrição de origens.
- **Content Security Policy (5 alertas)** — CSP Not Set, Wildcard, script-src unsafe-eval, style-src unsafe-inline, No Fallback.

> **Não aplicável:** WSTG-CONF-09/10/11 (File Permission, Subdomain Takeover, Cloud Storage) — sem estes elementos no ambiente de laboratório.

3.3 Identity Management (WSTG-IDNT)

Conta `admin@juice-sh.op` confirmada após SQLi (ver 3.4). Sem bloqueio de conta — o Fuzzer enviou 4 pedidos consecutivos sem throttling. WSTG-IDNT-04 (LDAP) não aplicável — sem LDAP no Juice Shop.

> **Nota (enunciado, ponto 2 Fase 2):** Problemas de Identity Management são falhas de lógica aplicacional sem padrões HTTP detetáveis. O ModSecurity não consegue detetar nem remediar estas questões.

3.4 Authentication Testing (WSTG-ATHN)

(d) Fuzz Attack ao Login

Campo `email` do POST `/rest/user/login` atacado com JSON (`application/json`) usando 4 payloads FuzzDB Offensive.

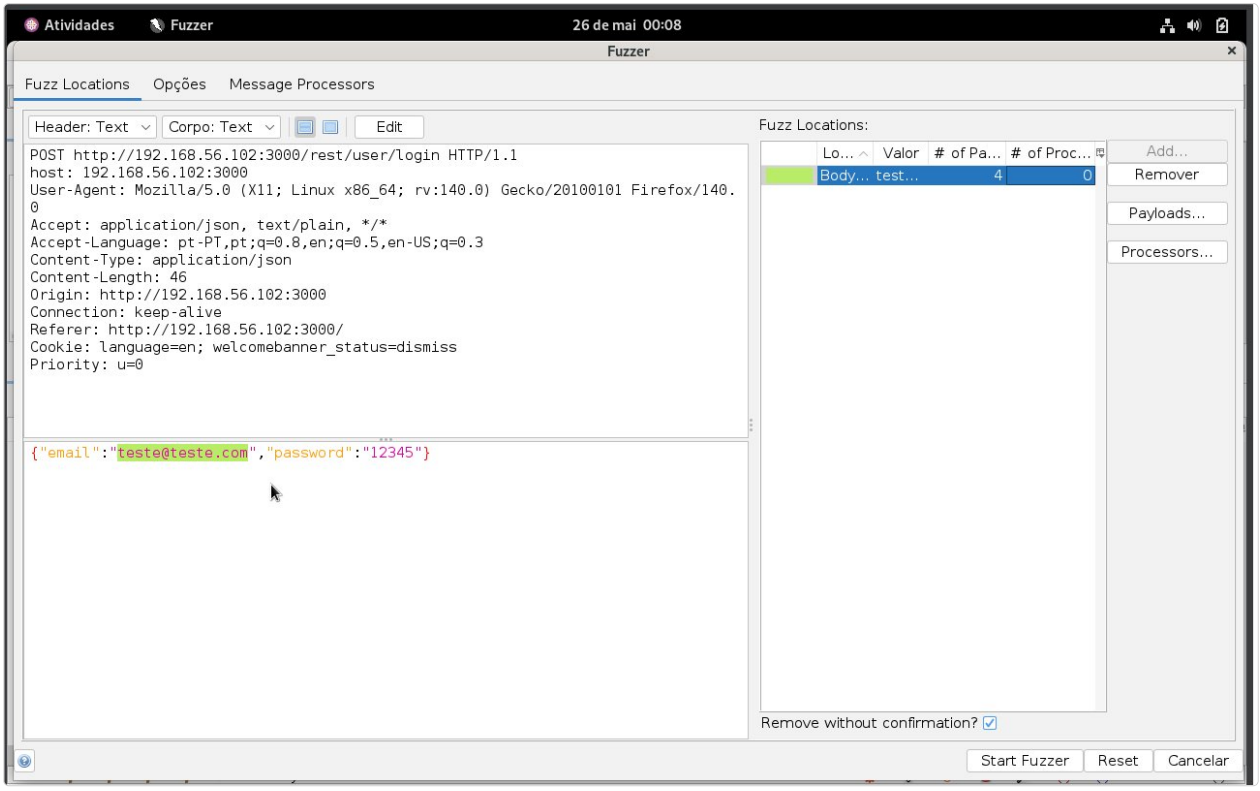


Figura 6 — Fuzzer: campo email selecionado (verde) no body JSON, 4 payloads carregados

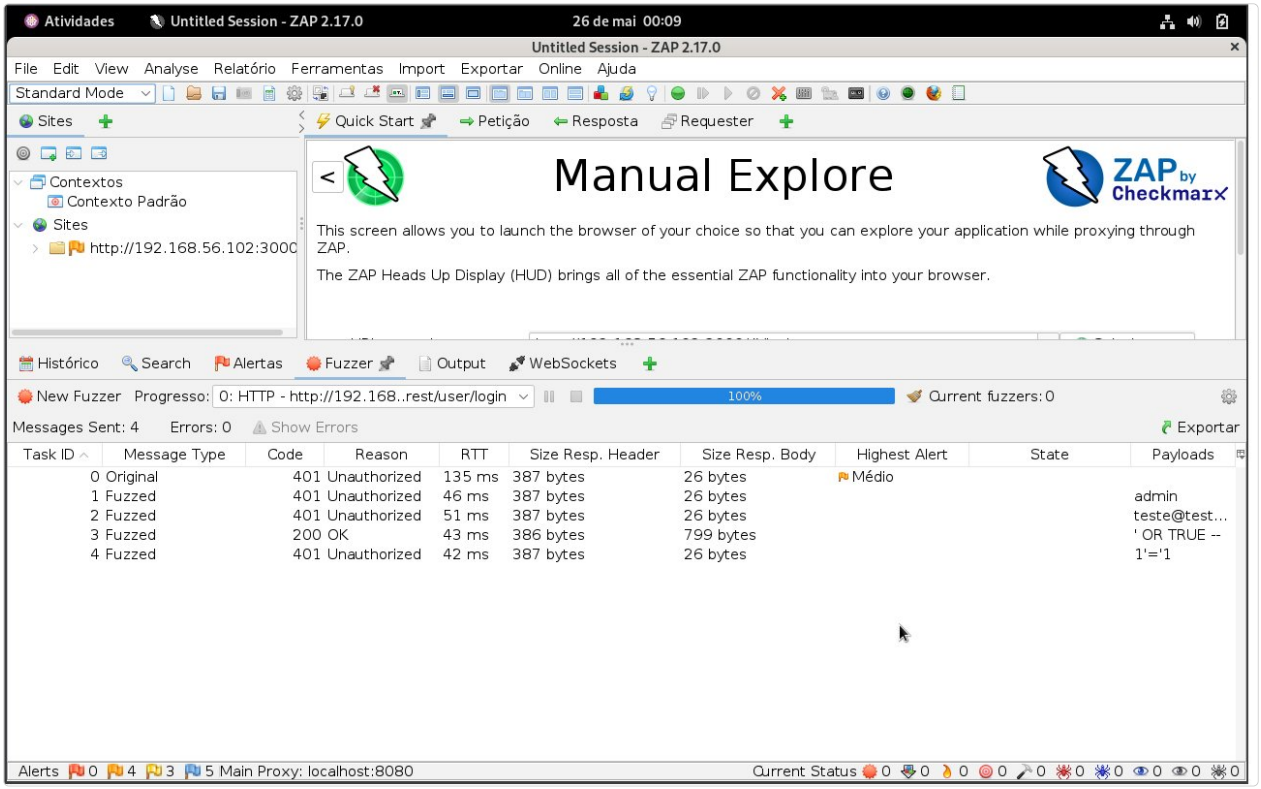


Figura 7 — Task 3 (' OR TRUE --) retorna 200 OK / 799 bytes — SQLi bem-sucedido; demais retornam 26 bytes

Task	Tipo	Payload (email)	HTTP	Body	Resultado
0	Original	teste@teste.com	401	26 B	Credenciais inválidas
1	Fuzzed	admin	401	26 B	Sem SQLi
2	Fuzzed	teste@teste.com	401	26 B	Sem SQLi
3	Fuzzed	`' OR TRUE --`	**200 OK**	**799 B**	**LOGIN ADMIN — SQL Injection**
4	Fuzzed	`1'='1`	401	26 B	Não explorado

(e) Teste Manual — Login como Admin

Payload `' OR TRUE --` torna a query SQL sempre verdadeira, devolvendo o primeiro registo (administrador). Challenge *«Login Admin»* resolvido.

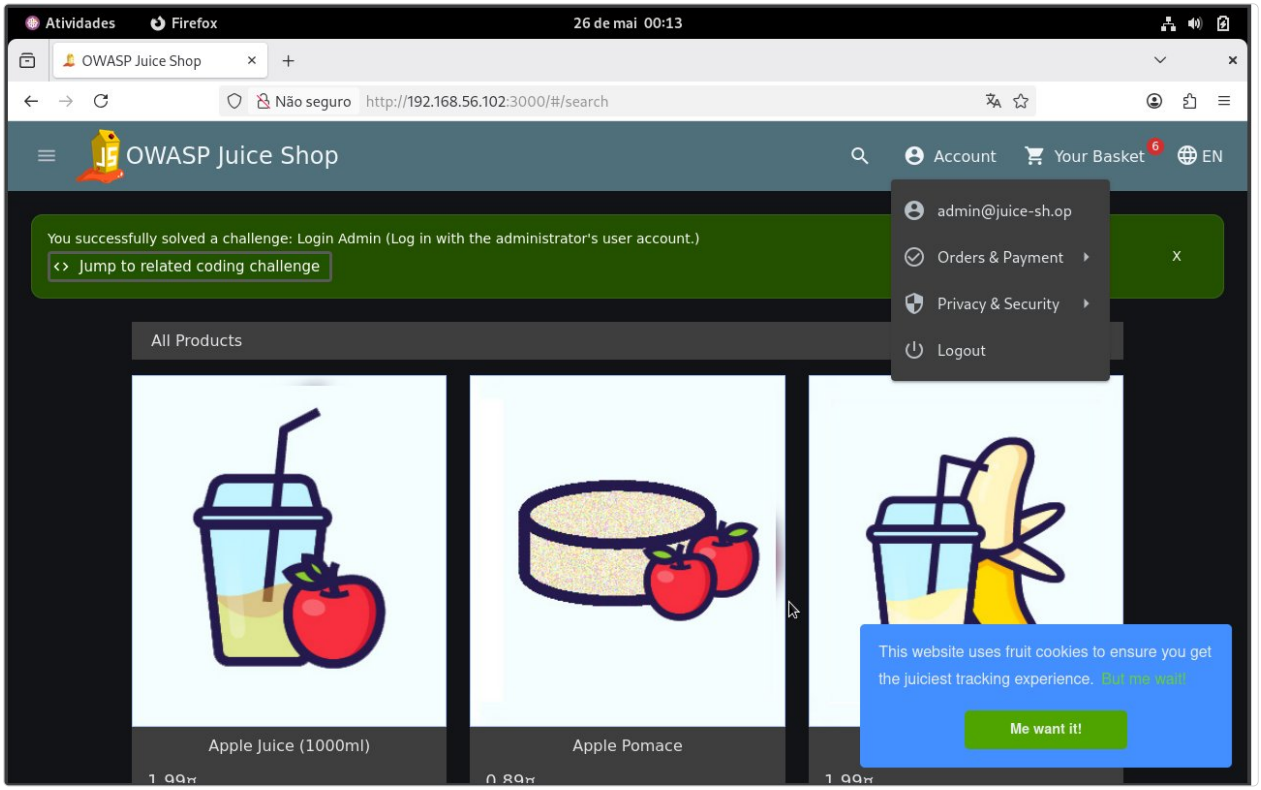


Figura 8 — Challenge resolvido: banner «Login Admin» + conta admin@juice-sh.op visível no menu

(f) Scan Ativo Autenticado

Session Properties configurado com utilizador obtido via SQLi. Scan autenticado: **19 864 pedidos**, expandindo de 10 para **26 alertas**.

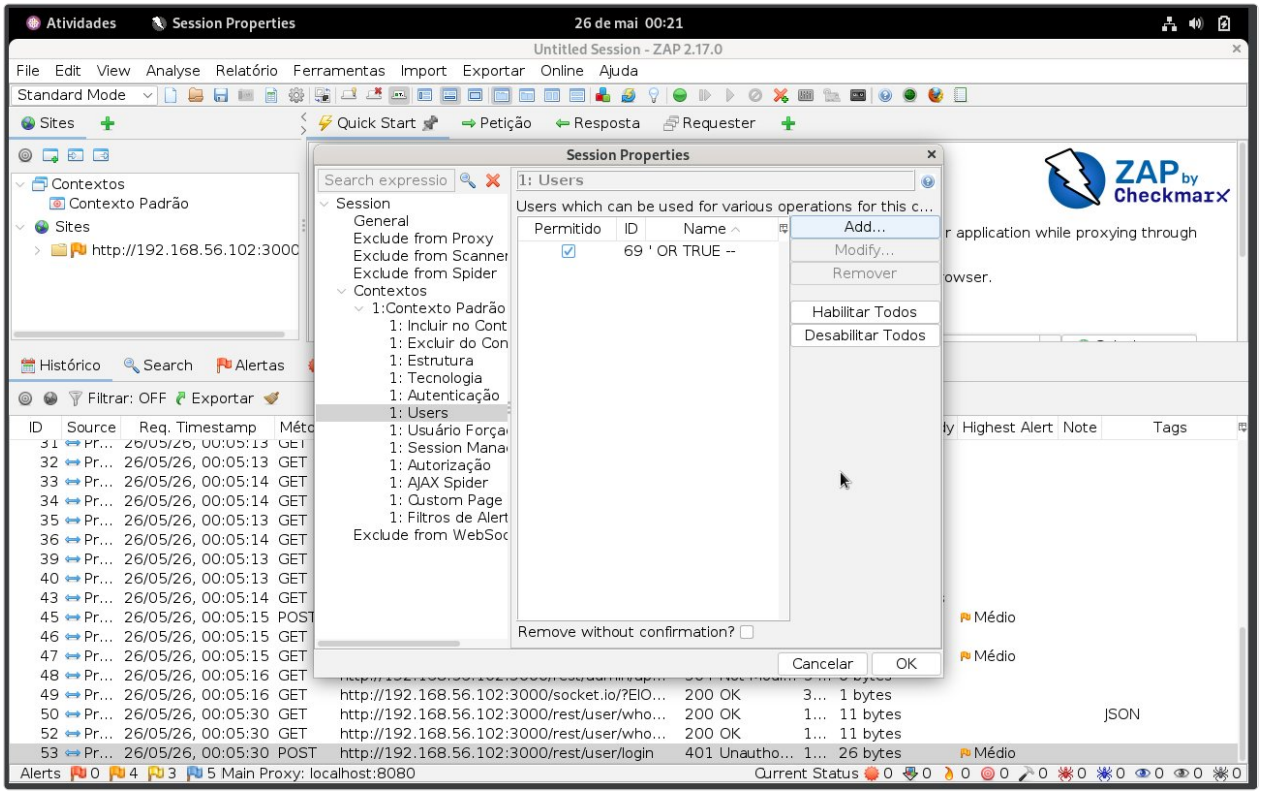


Figura 9 — Session Properties: utilizador SQLi configurado no contexto ZAP

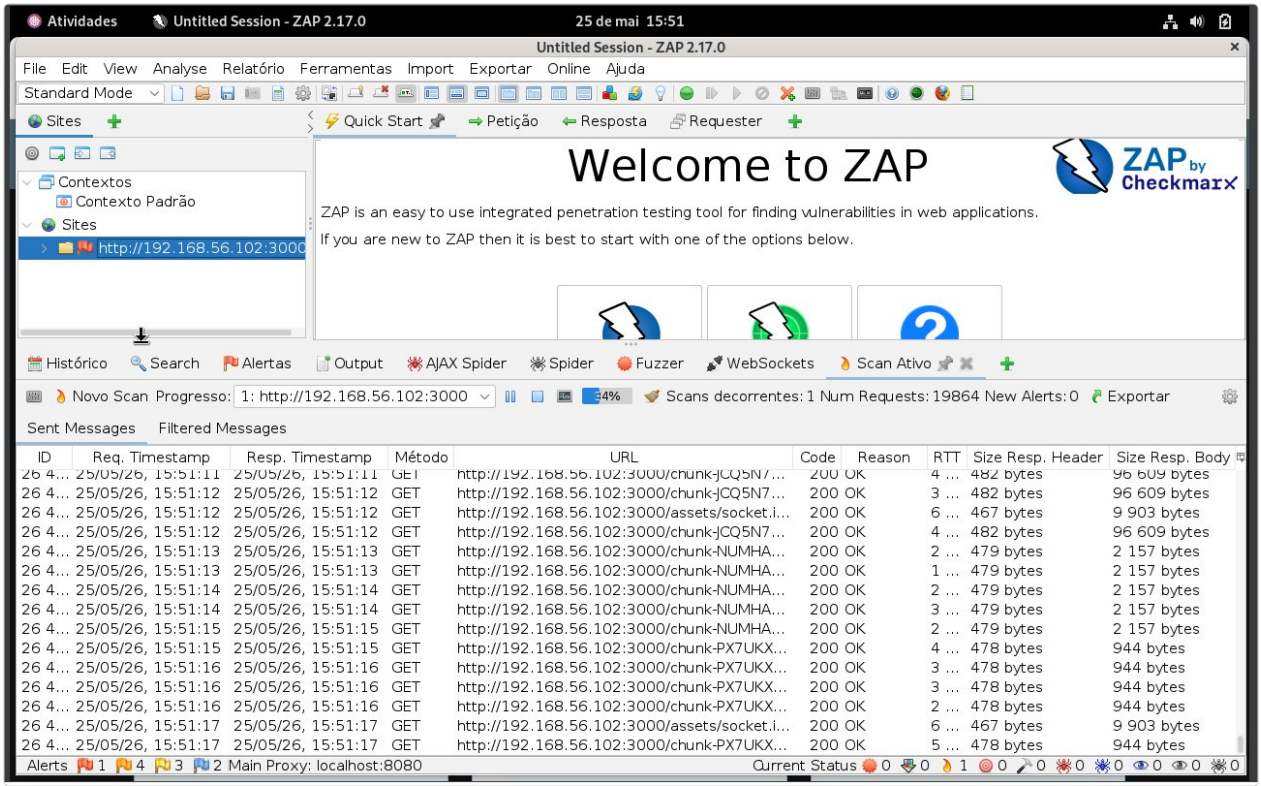


Figura 10 — Scan Autenticado: 19 864 pedidos, 84% — endpoints /rest/admin e /rest/user testados

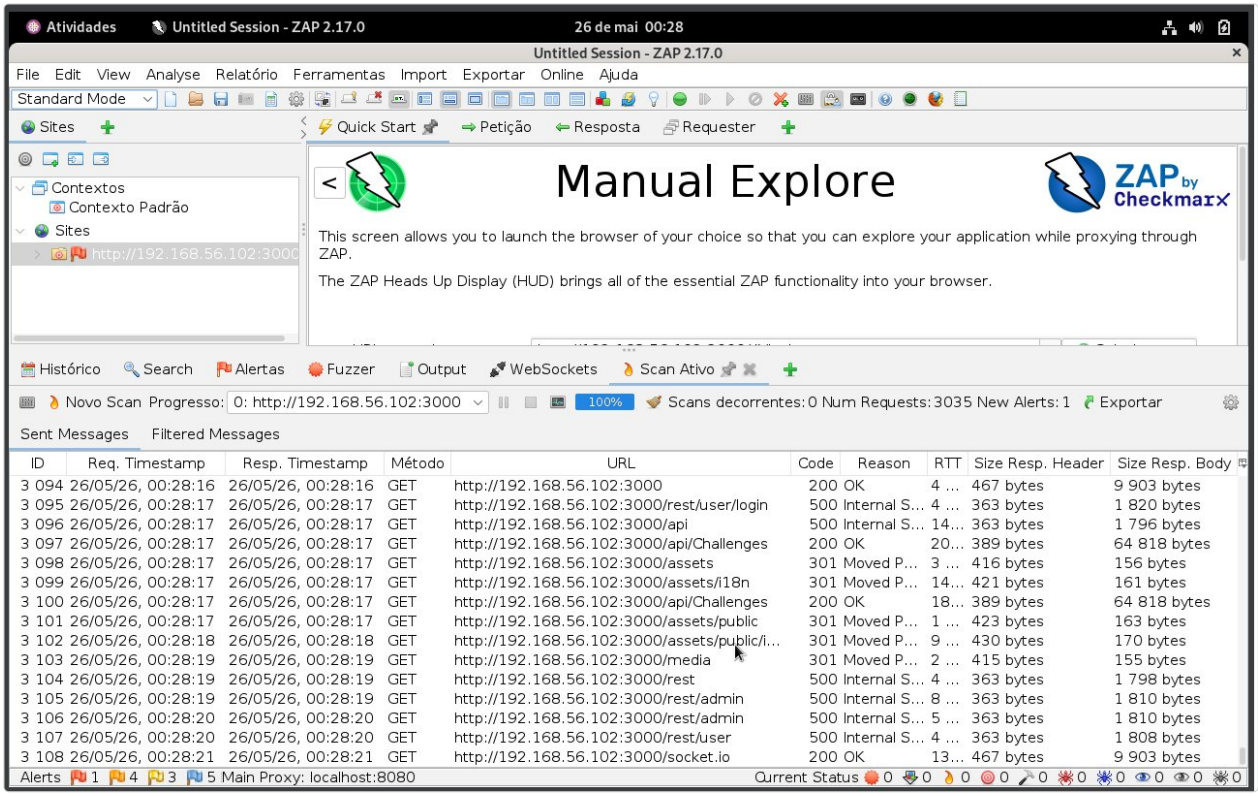


Figura 11 — Histórico: /rest/user/login, /rest/admin, /rest/user retornam 500 Internal Server Error

Vulnerabilidades adicionais identificadas:

- **Absence of Anti-CSRF Tokens (3):** Formulários de mutação de estado sem tokens de proteção — ataques CSRF possíveis.
- **JWT in Browser localStorage:** Token JWT acessível via JavaScript — vulnerável a roubo por XSS.
- **Application Error Disclosure (5) + Debug Error Messages:** Respostas 500 com stack traces da lógica interna.
- **Session Management Response Identified (2):** JWT stateless sem renovação ou invalidação forçada após login.

3.5 Authorization Testing (WSTG-ATHZ)

Após bypass via SQLi, o endpoint `/rest/admin` retorna 200 OK (Figura 11). Os controlos de autorização dependem exclusivamente da autenticação — contornada. Detetadas **3 instâncias de ausência de Anti-CSRF Tokens** em formulários de mutação de estado.

3.6 Session Management (WSTG-SESS)

- **Session ID in URL Rewrite (Systemic)** — tokens em URL, expostos em logs e Referer.
- **JWT em localStorage** — qualquer XSS permite roubo do token e impersonação.

- **Session Management Response Identified (2)** — JWT stateless sem renovação após login.
- **Cookies sem Secure/HttpOnly** — `language` e `welcomebanner_status` sem flags de segurança.

3.7 Input Validation (WSTG-INPV)

ZAP detetou **2 instâncias de SQL Injection** no scan ativo. Fuzzer confirmou com 200 OK / 799 bytes. Query vulnerável:

```
SELECT * FROM Users WHERE email = '' OR TRUE --' AND password = '...'
-- Condição sempre verdadeira → devolve o primeiro utilizador (administrador)
```

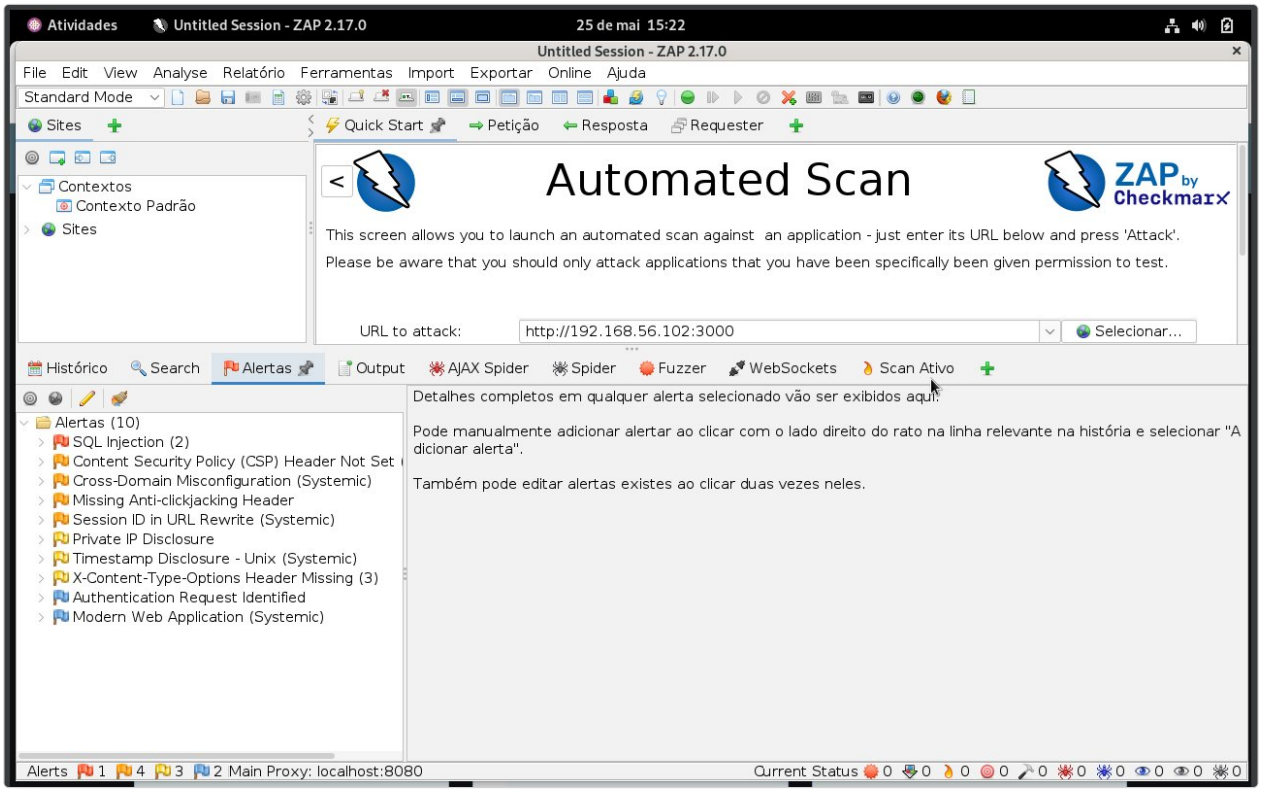


Figura 12 — 10 alertas do scan ativo: SQL Injection (2), CSP, Cross-Domain, Clickjacking, Session ID, Private IP, Timestamp, X-Content-Type (3)

> **Não aplicável:** WSTG-INPV-06 (LDAP — sem LDAP); WSTG-INPV-07 (XML — API usa JSON); WSTG-INPV-08 (SSI — não aplicável a Node.js).

3.8 Error Handling (WSTG-ERRH)

Scan autenticado: **5 instâncias de Application Error Disclosure** e *Debug Error Messages*. Figura 11 mostra `500 Internal Server Error` em `/rest/admin` e `/rest/user`, expondo detalhes da implementação interna.

3.9 Weak Cryptography (WSTG-CRYP) — Não Aplicável

> **Não aplicável** — enunciado, ponto 3: comunicação exclusivamente via HTTP sem TLS. Sem camada criptográfica a avaliar.

3.10 Business Logic (WSTG-BUSL) — Não Aplicável

> **Não aplicável** — enunciado, ponto 4: fora do âmbito deste trabalho.

3.11 Client Side Testing (WSTG-CLNT)

- **Missing Anti-clickjacking Header (3)** — sem `X-Frame-Options`; aplicação embebível em iframes maliciosos.
- **X-Content-Type-Options Missing (5)** — MIME sniffing possível; browser pode executar conteúdo mal-tipado como script.
- **Information Disclosure in Browser (2)** — JWT em `localStorage`.
- **Cross-Domain Misconfiguration (Systemic)** — CORS permissivo exposto ao cliente.

Fase 2 — Web Application Firewall

Com base nas vulnerabilidades da Fase 1, foi implementado um WAF com **Apache 2 + ModSecurity 2 + OWASP CRS** em modo bloqueio. Alterações em dois ficheiros: `httpd.conf` e `modsecurity.conf`.

Configuração Implementada

httpd.conf

```
# --- CONFIGURACAO DO WAF E PROXY (FASE 2) ---

# 1. Ocultar a versao do Apache e SO (Mitigar Information Disclosure)
ServerTokens Prod
ServerSignature Off

# 2. Inserir Headers de Seguranca (Mitigar Clickjacking e X-Content-Type)
Header always set X-Frame-Options "SAMEORIGIN"
Header always set X-Content-Type-Options "nosniff"

# 3. Configuracao do ModSecurity e OWASP CRS
<IfModule mod_security2.c>
    # Garantir que o motor esta em modo de bloqueio
    SecRuleEngine On

    # Ensinar o WAF a ler pacotes JSON (Obrigatorio para proteger o login do Juice Shop)
    SecRule REQUEST_HEADERS:Content-Type "application/json" \
        "id:'100001',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=JSON"

    # Mensagem de erro customizada para evidenciar o bloqueio
    ErrorDocument 403 "<h1>WAF: Acesso Bloqueado pelo ModSecurity/OWASP CRS</h1>"
</IfModule>

# 4. Reverse Proxy para o Juice Shop (Backend na porta 3000)
ProxyPreserveHost On
ProxyPass / http://127.0.0.1:3000/
ProxyPassReverse / http://127.0.0.1:3000/
```

Diretiva	Vulnerabilidade Visada	Justificação
`ServerTokens Prod` `ServerSignature Off`	Server Version Disclosure	Remove versão Apache e SO do header `Server`. Sem isto, o Apache exporia `Apache/2.x.x (CentOS)`.
`X-Frame-Options SAMEORIGIN`	Clickjacking (WSTG-CLNT-09)	Impede que a aplicação seja carregada em iframes de outras origens. SAMEORIGIN (vs DENY) permite iframes da mesma origem.
`X-Content-Type-Options nosniff`	MIME Sniffing (WSTG-CLNT-05)	Desativa inferência de tipo MIME pelo browser, prevenindo execução de conteúdo mal-tipado.
`SecRuleEngine On`	SQLi, XSS, OWASP Top 10	Ativa ModSecurity em modo bloqueio (enforcement). Sem isto, o motor só deteta mas não bloqueia.
Regra ID 100001 (JSON)	SQLi via API REST	**Crítica:** o Juice Shop usa `application/json` no login. Sem esta regra, o ModSecurity não analisa o corpo JSON e o CRS não deteta o SQLi no campo email.
`ErrorDocument 403`	Evidência de bloqueio	Mensagem customizada que confirma o bloqueio pelo ModSecurity/OWASP CRS (Figura 13).
`ProxyPass / ProxyPassReverse`	Encaminhamento	Proxy reverso: porta 80 → backend :3000.

modsecurity.conf

```
SecRuleEngine On
```

`SecRuleEngine On` garante modo bloqueio. O OWASP CRS é carregado via diretivas `Include`, ativando assinaturas de SQL Injection, XSS, RFI/LFI e Command Injection.

4.1 Information Gathering — Fase 2

Com `ServerTokens Prod`, o header `Server` deixou de expor versão e SO. O alerta *Server Leaks Version Information* foi eliminado. O Private IP Disclosure permanece — é exposto pelo código da aplicação, não pelo Apache.

4.2 Configuration and Deployment — Fase 2

Scan repetido contra `http://192.168.56.102` (porta 80 — WAF), política **Dev Standard**, AJAX Spider com Firefox: **13 alertas** (vs 26 na Fase 1).

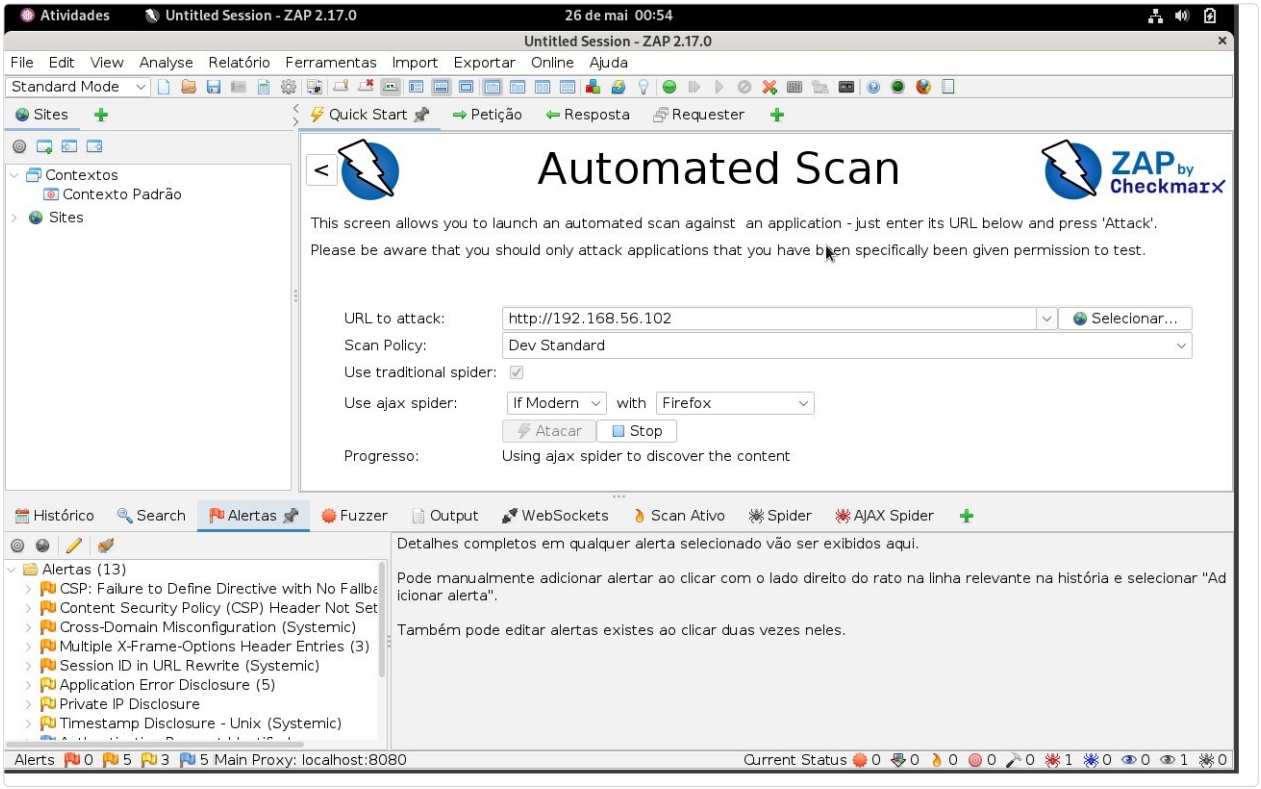


Figura 13 — Scan via WAF: http://192.168.56.102 (sem :3000), Dev Standard, AJAX Spider ativo, 13 alertas

`X-Frame-Options` e `X-Content-Type-Options` eliminaram os alertas correspondentes. O alerta *Multiple X-Frame-Options Header Entries* surge porque o Juice Shop já enviava este header e o Apache adicionou uma segunda instância — resolve-se com `Header always unset X-Frame-Options` antes do `set`. CORS e CSP permanecem — falhas no código da aplicação.

4.3 Identity Management — Fase 2

> **Sem ação requerida** — enunciado, ponto 2: o ModSecurity opera ao nível HTTP e não deteta falhas de lógica aplicacional.

4.4 Authentication Testing — Fase 2

SQL Injection — Bloqueado

O ataque que retornava 200 OK / 799 bytes retorna agora **403 Forbidden / 57 bytes**. O OWASP CRS detetou o padrão SQLi no JSON e bloqueou antes de atingir o backend.

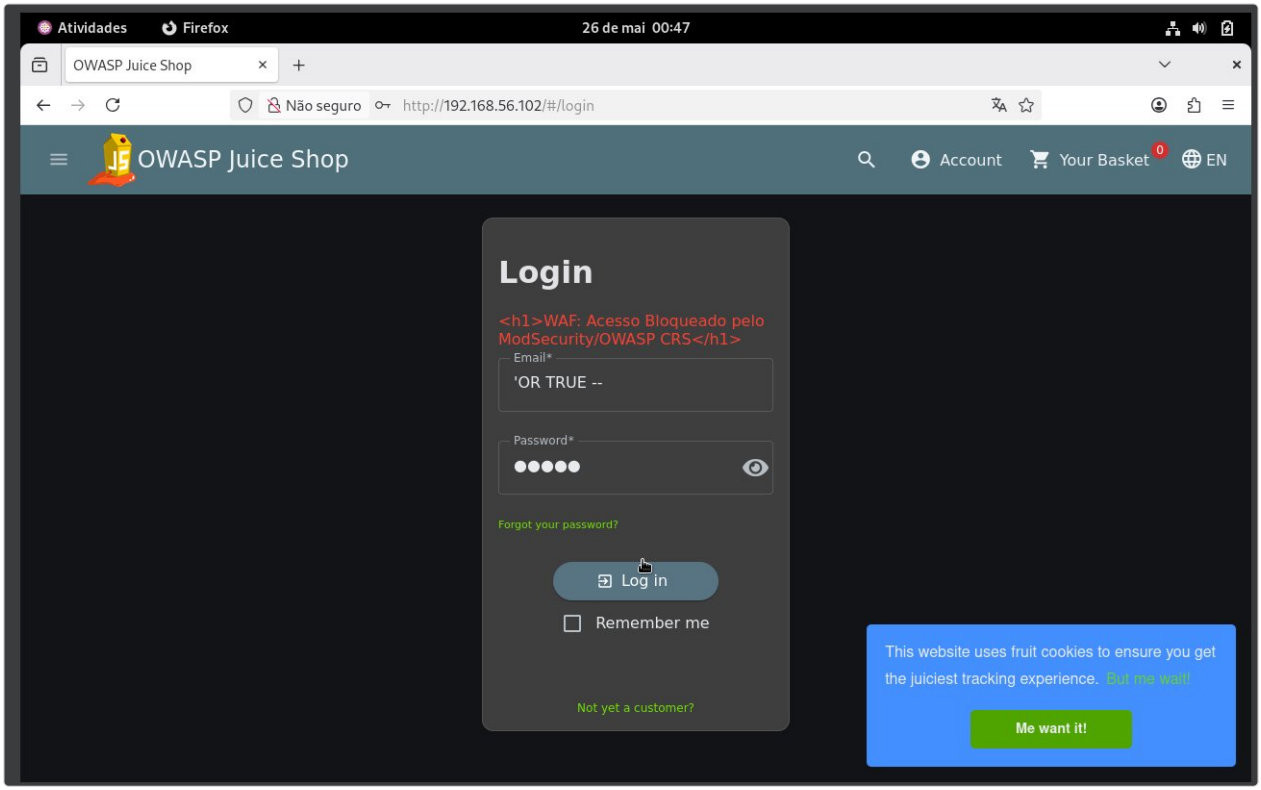


Figura 14 — WAF bloqueia SQLi: http://192.168.56.102, payload ' OR TRUE -- no email, mensagem «WAF: Acesso Bloqueado pelo ModSecurity/OWASP CRS»

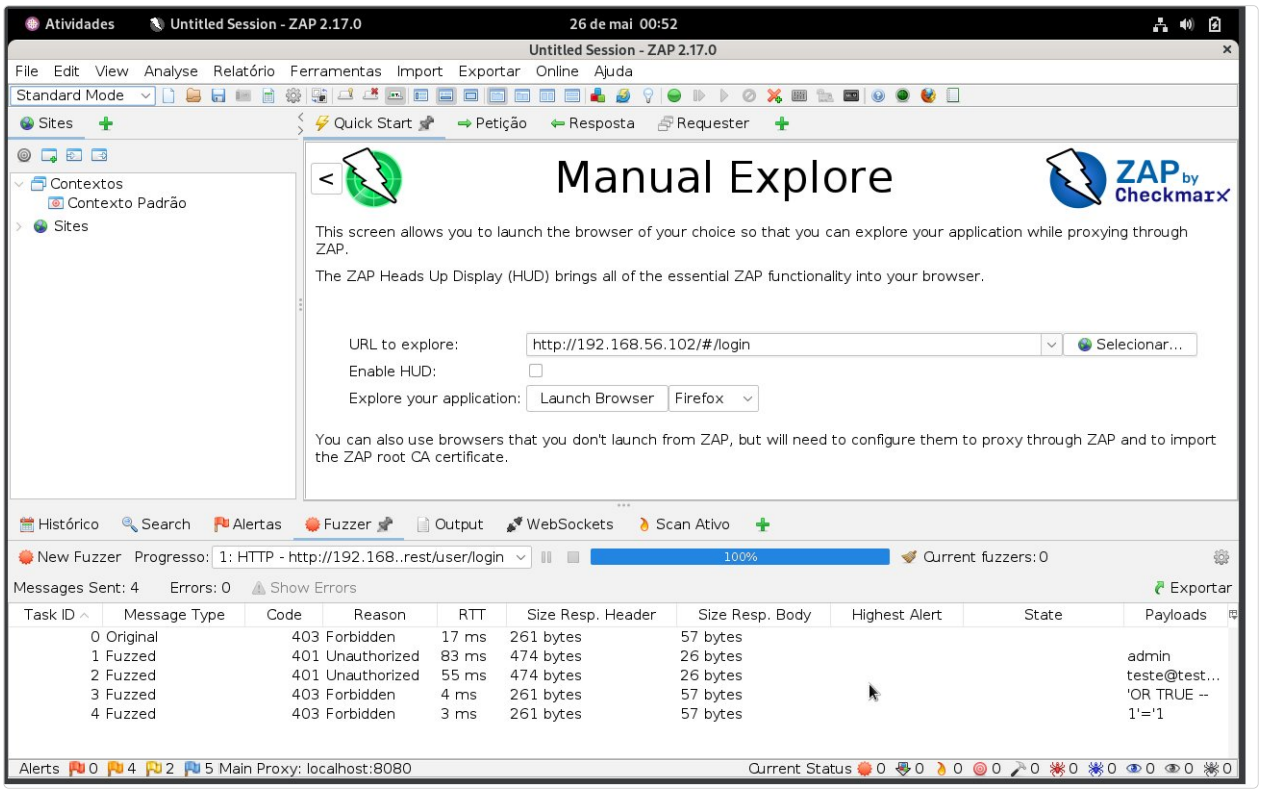


Figura 15 — Fuzzer via WAF: Tasks 3 e 4 (SQLi) → 403 Forbidden; Tasks 1 e 2 (sem SQLi) → 401 — WAF não bloqueia tráfego legítimo

Task	Payload	Fase 1	Fase 2 (WAF)	Resultado
0	Original	401	**403**	**Bloqueado — CRS interceta pedido no contexto do scan (enforcement mode)**
1	`admin`	401	401	Passado — sem padrão SQLi
2	`teste@teste.com`	401	401	Passado — sem padrão SQLi
3	`' OR TRUE --`	**200 OK (799B)**	**403 (57B)**	**BLOQUEADO — OWASP CRS**
4	`1'='1`	401	**403 (57B)**	**BLOQUEADO — OWASP CRS**

4.5 Authorization Testing — Fase 2

Com o SQLi bloqueado, o bypass de autenticação é impossível, tornando inviável o acesso indevido ao painel de administração. A cadeia de ataque SQLi → bypass auth → acesso admin é interrompida na primeira etapa. A ausência de Anti-CSRF Tokens permanece — falha de lógica aplicacional fora do alcance do ModSecurity.

4.6 Session Management — Fase 2

Os problemas de gestão de sessão (Session ID em URL, JWT em `localStorage`, cookies sem flags) são falhas do código do Juice Shop. O WAF opera ao nível HTTP e não tem visibilidade sobre o comportamento interno da aplicação — a remediação requer modificação do código-fonte.

4.7 Input Validation — Fase 2

Domínio de maior eficácia do WAF. O OWASP CRS bloqueou os payloads SQLi em ambos os vetores (browser e Fuzzer). A redução de 26 para 13 alertas (Figura 13) confirma que o ModSecurity bloqueia pedidos maliciosos antes de chegarem à aplicação. A regra JSON (ID 100001) foi determinante: sem ela, o CRS não analisaria o corpo `application/json` e o SQLi no campo email não seria detetado.

4.8 Error Handling — Fase 2

O WAF substitui as respostas de erro internas (500) pela página de bloqueio customizada (`ErrorDocument 403`), visível na Figura 14. Isto impede a exposição de stack traces para os vetores cobertos pelo CRS. Erros de pedidos legítimos continuam a ser geridos pela aplicação — requerem correção no código-fonte.

4.9 Weak Cryptography — Não Aplicável

> **Não aplicável** — comunicação HTTP sem TLS, idêntico à Fase 1. A implementação de HTTPS requereria certificados TLS e está fora do âmbito do WAF neste ambiente.

4.10 Business Logic — Não Aplicável

> **Não aplicável** — enunciado, ponto 4. O ModSecurity não tem capacidade para raciocinar sobre a lógica de negócio da aplicação.

4.11 Client Side Testing — Fase 2

Clickjacking mitigado com `X-Frame-Options: SAMEORIGIN`. MIME sniffing mitigado com `X-Content-Type-Options: nosniff`. JWT em `localStorage` e Cross-Domain Misconfiguration permanecem — falhas client-side fora do alcance do WAF.

Conclusões

A avaliação do OWASP Juice Shop revelou vulnerabilidades de severidade variada. A mais crítica — **SQL Injection no login** — permite contornar completamente a autenticação com o payload `' OR TRUE --`, obtendo acesso à conta de administrador. O Fuzzer do ZAP confirmou: Task 3 retorna 200 OK / 799 bytes, contra 26 bytes das respostas normais.

O WAF com **Apache 2 + ModSecurity + OWASP CRS** demonstrou eficácia clara. A regra JSON (ID 100001) foi essencial: sem ela, o CRS não analisaria os corpos `application/json` e o SQLi no campo email não seria detetado. Os headers de segurança eliminaram os alertas de clickjacking, MIME sniffing e divulgação de versão.

Limitação inerente: o WAF não resolve falhas de lógica aplicacional (CSRF, JWT em localStorage, CORS). A defesa em profundidade exige remediação em múltiplas camadas — código, configuração e infraestrutura.

Vulnerabilidade	Severidade	Estado Final (Fase 2)
SQL Injection — login bypass	**Alta**	**Mitigado — OWASP CRS (403); Figuras 14/15**
Ausência de rate limiting	**Alta**	**Parcial — CRS deteta padrões SQLi; sem lockout nativo**
Missing Anti-clickjacking Header	**Média**	**Mitigado — X-Frame-Options: SAMEORIGIN (Apache)**
X-Content-Type-Options Missing	Baixa	**Mitigado — nosniff (Apache)**
Server Version Disclosure	Baixa	**Mitigado — ServerTokens Prod (Apache)**
CSP ausente / incorreta	**Média**	**Parcial — CSP estrita quebra a SPA Angular; requer intervenção no código-fonte**
Absence of Anti-CSRF Tokens	**Média**	**Não mitigado — lógica aplicacional**
JWT em localStorage	**Média**	**Não mitigado — client-side**
Session ID em URL	**Média**	**Não mitigado — requer fix na aplicação**
Private IP Disclosure	**Média**	**Não mitigado — exposto pela aplicação**